Dr. Decode AI

Updated MVP Blueprint

Version: 1.1 | **Project:** Dr. Decode AI | **Approach:** Build-first, learn-by-doing, GitHub from Day 1

1. Project goal

Build an AI-powered web application that helps ordinary people understand medical documents in plain English.

Users select a document type, upload one supported document, and receive a structured explanation grounded in the document's actual contents.

Supported documents

1. Prescriptions
2. Laboratory test reports
3. Medical reports, including discharge summaries and written radiology reports

Supported formats: PDF, JPG, PNG and HEIC. Maximum upload size: 10 MB. One file per analysis.

Core principles

- Never invent medical facts or missing medication instructions.
- Preserve laboratory values, units and reference ranges.
- Separate document facts from general educational suggestions.
- Flag uncertainty, unreadable content and potentially concerning findings.
- Protect users' medical information and delete temporary files.
- Keep the initial application simple enough to build and maintain independently.

2. Technology stack

Component	Technology
Frontend	React, TypeScript, Vite
Frontend hosting	Cloudflare Pages
Backend	Python, FastAPI
Database	PostgreSQL
Database access	SQLAlchemy
Authentication	Supabase Auth
AI	Gemini API
Document processing	PDF text extraction and image processing
API testing	Postman
Version control	Git
Remote repository	GitHub
Deployment	Free-tier services, provider to be selected later

> We will verify current provider limits, data-handling terms and free-tier conditions before deployment. A zero-cost setup is the target, not a guarantee.

3. Architecture

The application will follow this logical flow:

```
React frontend  →  FastAPI backend  →  document extraction and validation
      →  Gemini API  →  structured explanation  →  frontend results
```

PostgreSQL will store the account-related application data, usage records and feedback. Supabase Auth will handle authentication.

Uploaded medical documents, extracted text and completed explanations will not be saved as application history.

The backend will be modular, but we will begin with a single FastAPI application. Additional services and infrastructure will be introduced only when needed.

4. Git and GitHub from Day 1

GitHub is a core part of this project, not a final publishing step.

Repository setup

Create a GitHub repository named `dr-decode-ai` and connect it to the local project from the beginning.

The initial repository will include:

- A README describing the project and setup.
- A `.gitignore` for secrets, virtual environments and generated files.
- The backend source code.
- An `.env.example` containing placeholder configuration values.
- Documentation and project structure as they develop.

How we will use Git

During each phase, you will:

1. Make a small, working change.
2. Inspect the changes using Git.
3. Stage the files you intend to include.
4. Create a meaningful commit.
5. Push your progress to GitHub at appropriate milestones.
6. Review the repository before moving to the next feature.

We will learn branches, pull requests and more advanced Git workflows when they become useful. We will not add unnecessary workflow complexity at the start.

GitHub security rules

- Never commit `.env` files, API keys, passwords or private credentials.
- Never commit real medical documents or sensitive user information.
- Keep generated files and virtual environments out of the repository.
- Review staged changes before committing.
- Use environment variables for secrets.

Learning resources

- **Git:** https://git-scm.com/doc
- **GitHub repository basics:** https://docs.github.com/en/repositories
- **GitHub authentication:** https://docs.github.com/en/authentication

5. Learning and implementation method

The goal is for you to write the code yourself and become comfortable building software through repeated practice.

Every phase will have its own Challenge PDF.

Structure of each Challenge PDF

#	Section	What it contains
1	**Mission**	What you are building and why.
2	**Requirements**	Features, inputs, outputs and expected behavior.
3	**Ordered coding tasks**	Small implementation steps that build on one another.
4	**Setup commands**	Necessary commands and environment configuration, without making you search for basic setup details.
5	**Just-in-time documentation**	Relevant official documentation linked beside the task that needs it.
6	**Hints**	Progressive clues, useful syntax and small examples without giving away the entire implementation.
7	**Debugging guidance**	Common errors and practical investigation techniques.
8	**GitHub milestones**	What to commit, when to push, and what each commit should represent.
9	**Testing scenarios**	Expected behavior for normal inputs, invalid inputs and edge cases.
10	**Repeat practice**	A small variation that reinforces a concept already learned.
11	**Completion criteria**	A clear definition of when the challenge is finished.

How we will work together

- You implement the solution.
- I explain concepts when they become relevant.
- You ask for hints or debugging help whenever you need it.
- I review your code and explain improvements.
- You fix and commit your work.
- We verify the acceptance criteria before proceeding.

We will not require extensive reading or memorization before coding. Repeated implementation will reinforce the concepts naturally.

6. Backend project structure

We will introduce files as they become necessary rather than creating the entire structure immediately.

The intended structure is:

```
dr-decode-ai/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/
│   │   ├── api/
│   │   ├── database/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   └── utils/
│   ├── requirements.txt
│   └── .env.example
├── frontend/
├── README.md
└── .gitignore
```

The API layer will coordinate requests. Services will contain business logic, and database modules will manage connections and queries.

7. API design

The initial API will expose:

Method	Endpoint	Purpose
GET	`/health`	Check backend availability
GET	`/api/v1/quota`	Return remaining attempts and reset time
POST	`/api/v1/analysis`	Upload and analyze a document
POST	`/api/v1/feedback`	Submit feedback

Protected endpoints will require a verified authentication token. FastAPI will validate access tokens and enforce authorization on the server.

Responses will use structured JSON. Errors will return appropriate HTTP statuses and safe, understandable messages.

8. Database design

The initial database will contain three main tables:

Table	Purpose
`profiles`	Application user identity and role.
`usage_records`	Daily processing-attempt counts.
`feedback`	Ratings, problem categories and optional comments.

We will use PostgreSQL with SQLAlchemy and bounded connection pooling.

For the MVP, database schema changes will be maintained manually in a single `schema.sql` file, as previously decided. Git history will provide a record of the changes to that file.

We will use transactions or atomic updates to prevent concurrent requests from bypassing quota limits.

9. Document processing and AI

The processing pipeline will:

1. Authenticate the user and validate the request.
2. Check file size, actual file type and selected document category.
3. Reject unsupported, password-protected or unsuitable documents.
4. Store the upload temporarily.
5. Extract text page by page where possible.
6. Preserve page references and original medical values.
7. Send relevant extracted content to Gemini as untrusted document data.
8. Validate the AI response against a strict Pydantic schema.
9. Check source references and important extracted values.
10. Return the structured explanation.
11. Delete temporary files after processing, including failure and cancellation paths where possible.

We will initially use straightforward extraction and document-grounded prompting rather than a vector database or a complex retrieval system.

Output templates

Document type	Contents
Laboratory reports	Summary, important findings, values and ranges, explanations, missing information, questions for the doctor, source references and appropriate warnings.
Prescriptions	Summary, written medication instructions, unclear details, questions for a doctor or pharmacist, source references and relevant warnings.
Medical reports	Summary, findings, explanations, general healthy-lifestyle suggestions where appropriate, missing information, questions for the doctor and source references.

> Gemini output must not be treated as medically correct merely because it passes schema validation. Safety rules must not rely on Gemini alone.

10. Security and privacy

- Use Supabase Auth for sign-in, email verification and password recovery.
- Require verified email before document processing.
- Enforce authorization and rate limits in the backend.
- Keep API keys and credentials in environment variables.
- Use HTTPS in deployed environments.
- Avoid logging medical content or credentials.
- Delete temporary documents after processing.
- Do not retain document history or full explanations.
- Do not attach medical files automatically to feedback.
- Review Gemini's applicable API data-handling terms before sending real medical documents.

> Use synthetic or non-sensitive documents during development.

11. Quotas and error handling

> The planned limit is **five processing attempts per account per day**, resetting at midnight in the Europe/Prague timezone.

The system will also enforce the 10 MB upload limit, basic account/IP rate limits and processing timeouts.

Transient AI failures may receive one automatic retry. Invalid structured output will be rejected rather than automatically repaired. Temporary failures should offer a manual retry where appropriate.

Database recovery attempts will be bounded. Logs will contain only minimal, sanitized diagnostics.

12. Frontend design

The frontend will contain:

- A public landing page.
- Signup, sign-in and password recovery.
- Document-type selection and upload.
- A loading state with no fake progress percentage.
- A results page showing the original document beside the explanation on desktop and vertically on mobile.
- Source references and basic PDF navigation.
- A downloadable explanation PDF.
- Feedback controls and an option to analyze another document.

The original uploaded document will not be downloadable from the application. Starting another analysis will clear the previous result.

13. Development roadmap

> **Phase 1 — Backend foundation**
>
> Create the GitHub repository, establish the local project, configure the Python environment, create a FastAPI application and implement `/health`.
>
> **Deliverable:** A working local backend, a successful health check, and a GitHub repository with meaningful commits.

> **Phase 2 — Database and authentication**
>
> Configure PostgreSQL, SQLAlchemy, the initial schema and Supabase Auth.
>
> **Deliverable:** Protected endpoints and verified user authentication.

> **Phase 3 — Upload and extraction**
>
> Implement file validation, temporary storage, PDF/image extraction and cleanup.
>
> **Deliverable:** A validated document can be converted into extracted text with page references.

> **Phase 4 — Gemini integration**
>
> Implement prompts, structured output schemas, validation and transient-error handling.
>
> **Deliverable:** Supported documents produce structured explanations.

Phase 5 — Quota and security

Implement usage limits, rate limiting, privacy-safe logging and robust failure handling.

Deliverable: The backend enforces the principal security and usage rules.

Phase 6 — API completion

Finish and manually test the analysis, quota, feedback and health endpoints using Postman.

Deliverable: The backend workflow works without a frontend.

Phase 7 — Frontend

Build the React + TypeScript interface and connect it to the backend.

Deliverable: Users can complete the entire workflow through the browser.

Phase 8 — Testing and deployment

Manually validate normal cases, failures, security boundaries and cleanup. Review provider limits and privacy terms before deploying to free-tier services.

Deliverable: A portfolio-ready MVP, documented on GitHub, with reproducible setup instructions.

14. Definition of done

The project is ready when a verified user can upload a supported document, receive an explanation grounded in the source, inspect source references, download the explanation and submit feedback.

The application must also protect accounts, enforce quotas, handle invalid inputs and service failures safely, and delete temporary medical files.

The GitHub repository must contain the source code, project documentation, setup instructions and appropriate commit history without secrets or private medical data.

15. Final working agreement

You are the developer. I am your mentor.

You write the code, make the implementation decisions and commit the work. I provide challenge briefs, targeted documentation, hints, debugging support and code reviews.

We will learn by repeatedly building real features, not by trying to memorize every concept before starting.

The immediate next step is the Phase 1 Challenge PDF, designed around this build-first method and GitHub from the very beginning.